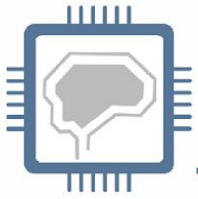


***Inteligência Artificial
e Computacional***

Disciplina: Linguagens de Programação para Inteligência Artificial

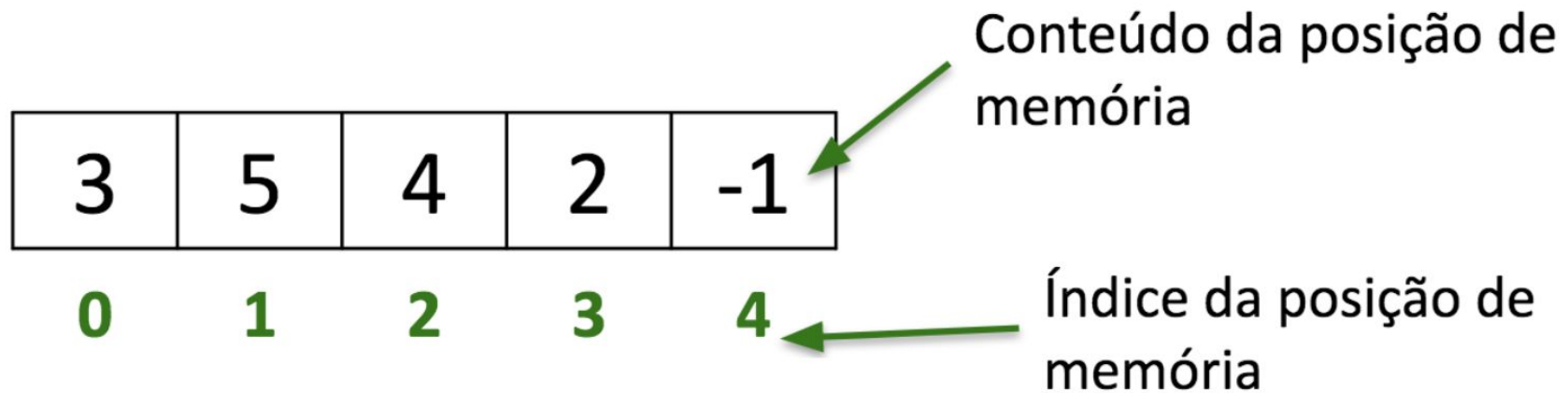
Aula 3: Arranjos

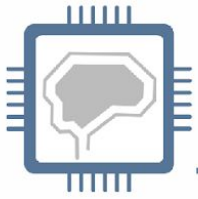
Profa.: Sabrina de A. Silveira



Arranjos

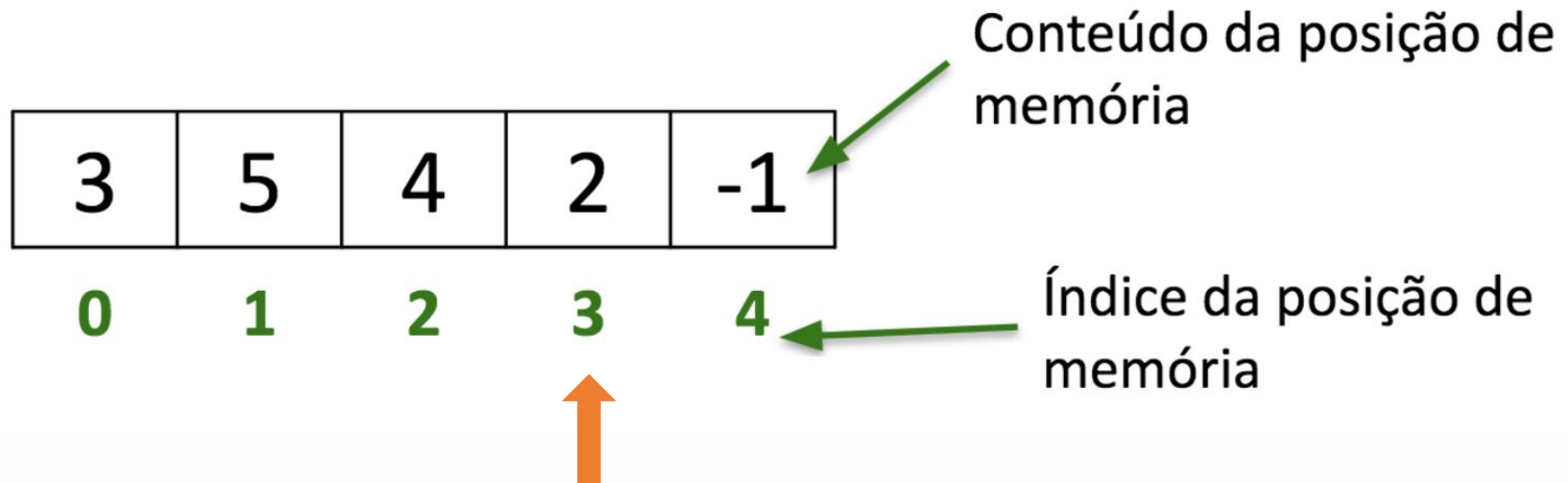
- Um arranjo ou array é um conjunto de posições reservadas na memória do computador para armazenar um conjunto de elementos de um mesmo tipo.
- Cada posição do arranjo é indexada por um número inteiro, começando a partir do 0.



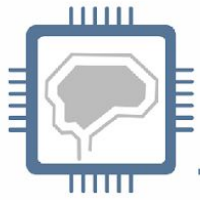


Arranjos

- Para acessar o elemento na quarta posição do arranjo usamos a sintaxe `v[3]`.
- O conteúdo da posição `v[3]` é 2.

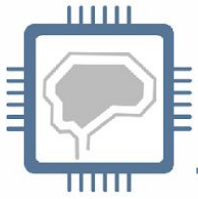


- Os arranjos podem ser de 1, 2, 3 ou n dimensões.



Vetores

- Neste curso, usamos a palavra **vetor** para nos referir a arranjos de uma única dimensão.
- O conceito de vetor é semelhante ao da Matemática.
- A biblioteca **Numpy** do Python será utilizada para criar e manipular arranjos numéricos.
- Essa biblioteca facilita a criação e inicialização de arranjos e é também mais semelhante a como outras linguagens de programação tratam arranjos.

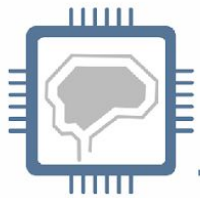


Vetores 1

Neste exemplo a biblioteca Numpy é utilizada para criar e inicializar vetores. Observe os comentários no programa. Eles ajudam a explicar o que é feito em cada linha.

Importa a biblioteca Numpy antes de usá-la. Aqui chamamos a biblioteca Numpy de np

```
→ import numpy as np # np eh como chamamos o numpy para ficar sucinto  
  
notas = np.empty(5) # vetor de float, pode ter lixo na memoria  
print("notas = ", notas)
```



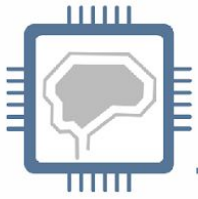
Vetores 1

Neste exemplo a biblioteca Numpy é utilizada para criar e inicializar vetores. Observe os comentários no programa. Eles ajudam a explicar o que é feito em cada linha.

```
import numpy as np # np eh como chamamos o numpy para ficar sucinto
```

```
➔ notas = np.empty(5) # vetor de float, pode ter lixo na memoria  
print("notas = ", notas)
```

Cria um vetor do tipo float, com 5 posições e não inicializa o vetor.



Vetores 1

Neste exemplo a biblioteca Numpy é utilizada para criar e inicializar vetores. Observe os comentários no programa. Eles ajudam a explicar o que é feito em cada linha.

```
import numpy as np # np eh como chamamos o numpy para ficar sucinto
```

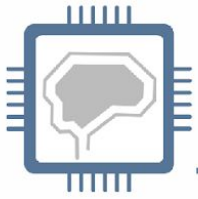
```
notas = np.empty(5) # vetor de float, pode ter lixo na memoria
```



```
print("notas = ", notas)
```

Imprime o conteúdo do vetor na tela

```
notas = [0.00000000e+000 2.00000012e+000 1.48219694e-323 0.00000000e+000  
0.00000000e+000]
```



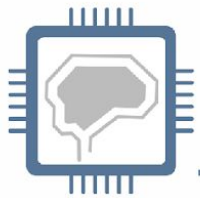
Vetores 2

Neste exemplo a biblioteca Numpy é utilizada para criar e inicializar vetores. Observe os comentários no programa. Eles ajudam a explicar o que é feito em cada linha.

```
import numpy as np # np eh como chamamos o numpy para ficar sucinto
```

```
→ notas2 = np.zeros(5) # vetor de float incializado com 0  
print("notas2 = ",notas2)
```

Cria um vetor do tipo float, com 5 posições e inicializa as posições do vetor com o valor 0.



Vetores 2

Neste exemplo a biblioteca Numpy é utilizada para criar e inicializar vetores. Observe os comentários no programa. Eles ajudam a explicar o que é feito em cada linha.

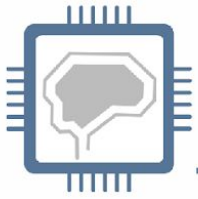
```
import numpy as np # np eh como chamamos o numpy para ficar sucinto
```

```
notas2 = np.zeros(5) # vetor de float incializado com 0
```

```
→ print("notas2 = ", notas2)
```

Imprime o conteúdo do vetor na tela

```
notas2 = [0. 0. 0. 0. 0.]
```



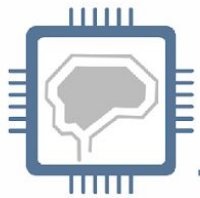
Vetores 3

Neste exemplo a biblioteca Numpy é utilizada para criar e inicializar vetores. Observe os comentários no programa. Eles ajudam a explicar o que é feito em cada linha.

```
import numpy as np_# np eh como chamamos o numpy para ficar sucinto
```

```
→ idades = np.zeros(5, dtype=int)_# vetor de inteiros inicializado com 0  
print("idades = ", idades)
```

Cria um vetor do tipo inteiro, com 5 posições e inicializa as posições do vetor com o valor 0.



Vetores 3

Neste exemplo a biblioteca Numpy é utilizada para criar e inicializar vetores. Observe os comentários no programa. Eles ajudam a explicar o que é feito em cada linha.

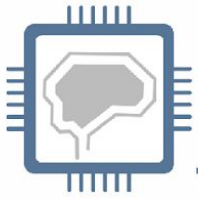
```
import numpy as np # np eh como chamamos o numpy para ficar sucinto
```

```
idades = np.zeros(5, dtype=int) # vetor de inteiros inicializado com 0
```

```
➔ print("idades = ", idades)
```

Imprime o conteúdo do vetor na tela

```
idades = [0 0 0 0 0]
```



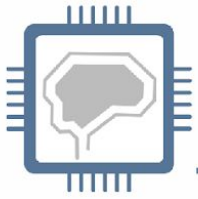
Vetores 4

Neste exemplo a biblioteca Numpy é utilizada para criar e inicializar vetores. Observe os comentários no programa. Eles ajudam a explicar o que é feito em cada linha.

```
import numpy as np # np eh como chamamos o numpy para ficar sucinto
```

```
→ contador = np.array([4, 3, 2, 1]) # inicializa na criacao  
print("contador = ", contador)
```

Cria um vetor com 4 posições e inicializa as posições do vetor com os valores 4, 3, 2, 1.



Vetores 4

Neste exemplo a biblioteca Numpy é utilizada para criar e inicializar vetores. Observe os comentários no programa. Eles ajudam a explicar o que é feito em cada linha.

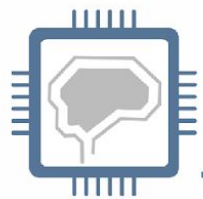
```
import numpy as np # np eh como chamamos o numpy para ficar sucinto
```

```
contador = np.array([4, 3, 2, 1]) # inicializa na criacao
```

```
→ print("contador = ", contador)
```

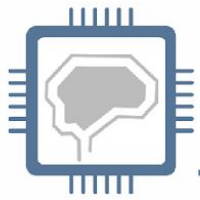
Imprime o conteúdo do vetor na tela. Observe que o vetor é do tipo inteiro.

```
contador = [4 3 2 1]
```



Vetores

Quando usamos arranjos, é muito comum precisarmos percorrer o mesmo, realizando algum tipo de tarefa, como operações Matemáticas, impressão formatada na tela, dentre outros.



Vetores

Programa pede ao usuário que informe o número de elementos do vetor.



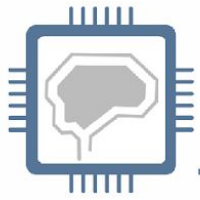
```
import numpy as np

n = int(input('Informe o número de elementos: '))
A = np.empty(n) # cria arranjo

# le do teclado
for i in range(0, n):
    A[i] = float(input('Informe o %dº elemento: ' % (i+1)))

# multiplica o vetor por 3
for i in range(0, n):
    A[i] = A[i] * 3

# escreve na tela com formatacao
print("Imprimindo vetor na tela...")
for i in range(0, n):
    print('%5.2f' % A[i], end='')
print()
```



Vetores

Cria vetor float de tamanho n



```
import numpy as np
```

```
n = int(input('Informe o número de elementos: '))
```

```
A = np.empty(n) # cria arranjo
```

```
# le do teclado
```

```
for i in range(0, n):
```

```
    A[i] = float(input('Informe o %dº elemento: ' % (i+1)))
```

```
# multiplica o vetor por 3
```

```
for i in range(0, n):
```

```
    A[i] = A[i] * 3
```

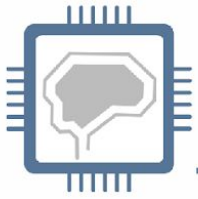
```
# escreve na tela com formatacao
```

```
print("Imprimindo vetor na tela...")
```

```
for i in range(0, n):
```

```
    print('%5.2f' % A[i], end='')
```

```
print()
```



Vetores

Percorre o vetor A criado e pede que o usuário informe cada um dos n valores

```
import numpy as np
```

```
n = int(input('Informe o número de elementos: '))
```

```
A = np.empty(n) # cria arranjo
```

```
# le do teclado
```

```
for i in range(0, n):
```

```
    A[i] = float(input('Informe o %dº elemento: ' % (i+1)))
```

```
# multiplica o vetor por 3
```

```
for i in range(0, n):
```

```
    A[i] = A[i] * 3
```

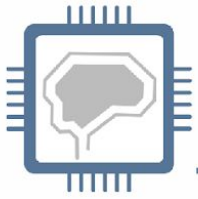
```
# escreve na tela com formatacao
```

```
print("Imprimindo vetor na tela...")
```

```
for i in range(0, n):
```

```
    print('%5.2f' % A[i], end='')
```

```
print()
```



Vetores

Percorre o vetor A e multiplica o conteúdo de cada uma das posições por 3.

```
import numpy as np
```

```
n = int(input('Informe o número de elementos: '))
```

```
A = np.empty(n) # cria arranjo
```

```
# le do teclado
```

```
for i in range(0, n):
```

```
    A[i] = float(input('Informe o %dº elemento: ' % (i+1)))
```

```
# multiplica o vetor por 3
```

```
for i in range(0, n):
```

```
    A[i] = A[i] * 3
```

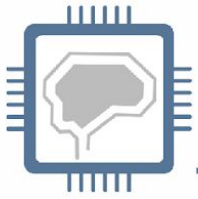
```
# escreve na tela com formatacao
```

```
print("Imprimindo vetor na tela...")
```

```
for i in range(0, n):
```

```
    print('%5.2f' % A[i], end='')
```

```
print()
```



Vetores

Percorre o vetor A e imprime cada posição na tela com comprimento 5 e 2 casas decimais.

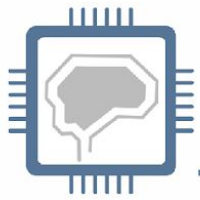
```
import numpy as np

n = int(input('Informe o número de elementos: '))
A = np.empty(n) # cria arranjo

# le do teclado
for i in range(0, n):
    A[i] = float(input('Informe o %dº elemento: ' % (i+1)))

# multiplica o vetor por 3
for i in range(0, n):
    A[i] = A[i] * 3

# escreve na tela com formatacao
print("Imprimindo vetor na tela...")
for i in range(0, n):
    print('%5.2f' % A[i], end='')
print()
```



Vetores

```
import numpy as np

n = int(input('Informe o número de elementos: '))
A = np.empty(n) # cria arranjo

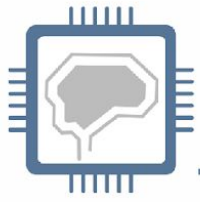
# le do teclado
for i in range(0, n):
    A[i] = float(input('Informe o %dº elemento: ' % (i+1)))

# multiplica o vetor por 3
for i in range(0, n):
    A[i] = A[i] * 3

# escreve na tela com formatacao
print("Imprimindo vetor na tela...")
for i in range(0, n):
    print('%5.2f' % A[i], end='')
print()
```

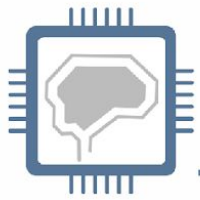
Saída do programa

```
Informe o número de elementos: 4
Informe o 1º elemento: 1
Informe o 2º elemento: 2.5
Informe o 3º elemento: 1
Informe o 4º elemento: 3
Imprimindo vetor na tela...
 3.00  7.50  3.00  9.00
```



Strings

Os strings em Python são vetores de caracteres. Podemos acessar os elementos de strings utilizando colchetes, do mesmo modo que fizemos para os vetores. Podemos também percorrer os strings e executar diversas tarefas.



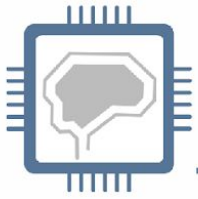
Strings

Inicializa o string `a` com "Diga Olá!" e imprime o elemento de índice 1 de `a`.

```
a = "Diga Olá!"  
print("Caracter de índice 1...")  
print(a[1]) # acessa elemento do string  
print()
```

```
# percorre elemento do string 1 a 1  
print("Imprime string...")  
for x in a:  
    print(x, end="")  
print("\n")
```

```
# percorre string com passo 2  
print("Imprime string...")  
for i in range(0, len(a), 2):  
    print(a[i], end="")
```



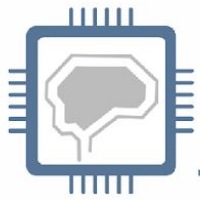
Strings

Percorre o string `a` e imprime cada um de seus caracteres na tela com separador vazio `""`. Imprime nova linha ao final

```
a = "Diga Olá!"  
print("Caracter de índice 1...")  
print(a[1]) # acessa elemento do string  
print()
```

```
# percorre elemento do string 1 a 1  
print("Imprime string...")  
{ for x in a:  
    print(x, end="")  
  print("\n")
```

```
# percorre string com passo 2  
print("Imprime string...")  
for i in range(0, len(a), 2):  
    print(a[i], end="")
```



Strings

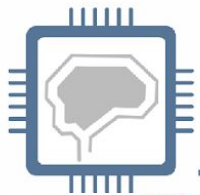
Percorre o string **a** começando do índice 0 e indo até o final de **a**, com passo 2. Imprime cada caracter com separador vazio ao final

```
a = "Diga Olá!"  
print("Caracter de índice 1...")  
print(a[1]) # acessa elemento do string  
print()
```

```
# percorre elemento do string 1 a 1  
print("Imprime string...")  
for x in a:  
    print(x, end="")  
print("\n")
```

len(a) é uma função que retorna o tamanho do string **a**

```
# percorre string com passo 2  
print("Imprime string...")  
for i in range(0, len(a), 2):  
    print(a[i], end="")
```



Strings

```
a = "Diga Olá!"  
print("Caracter de índice 1...")  
print(a[1]) # acessa elemento do string  
print()
```

```
# percorre elemento do string 1 a 1  
print("Imprime string...")  
for x in a:  
    print(x, end="")  
print("\n")
```

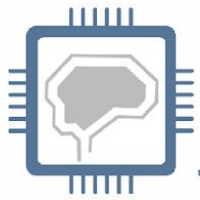
```
# percorre string com passo 2  
print("Imprime string...")  
for i in range(0, len(a), 2):  
    print(a[i], end="")
```

Saída do programa

Caracter de índice 1...
i

Imprime string...
Diga Olá!

Imprime string...
Dg l!



Matrizes

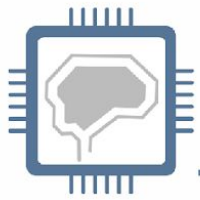
- As matrizes são arranjos de 2 dimensões. Podemos pensar nas matrizes como um diagrama com linhas e colunas.
- Para acessar um elemento, usamos a sintaxe `v[1][2]`, que indica que queremos acessar o elemento na linha de índice 1 e na coluna de índice 2, lembrando que os índices começam do 0.

colunas

linhas →

Índices	0	1	2	3
0	4	1	2.5	2
1	-1	0	3	-2
2	0.2	6	5	-0.1

Os dados da matriz estão na memória do computador



Matrizes

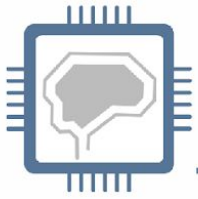
- O elemento $v[1][2]$ tem o valor 3 na matriz apresentada.
- Essa matriz possui 3 linhas e 4 colunas, totalizando 12 elementos.
- Em verde mostramos os índices. Eles não fazem parte dos dados contidos na matriz

colunas

linhas →

Índices	0	1	2	3
0	4	1	2.5	2
1	-1	0	3	-2
2	0.2	6	5	-0.1

Os dados da matriz estão na memória do computador



Matrizes

Pede ao usuário que digite o número de linhas (nl) e o número de colunas (nc) para criar uma matriz.

```
import numpy as np
```

```
# le dimensões da matriz
```

```
nl = int(input('Informe o número de linhas : '))
```

```
nc = int(input('Informe o número de colunas: '))
```

```
# cria matriz
```

```
A = np.empty((nl, nc))
```

```
# le do teclado
```

```
for i in range(0, nl): # linhas vao de 0 a nl-1
```

```
    for j in range(0, nc): # colunas vao de 0 a nc-1
```

```
        s = 'Informe o elemento A[%d][%d]: ' % (i, j)
```

```
        A[i][j] = float(input(s))
```

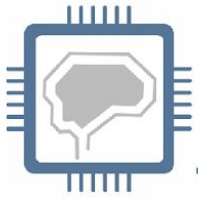
```
print("Imprime matriz...")
```

```
for i in range(0, nl):
```

```
    for j in range(0, nc):
```

```
        print('%5.1f' % (A[i][j]), end='')
```

```
    print() # imprime quebra de linha
```



Matrizes

Cria uma matriz com as dimensões nl e nc informadas pelo usuário

```
import numpy as np
```

```
# le dimensões da matriz
```

```
nl = int(input('Informe o número de linhas : '))
```

```
nc = int(input('Informe o número de colunas: '))
```

```
# cria matriz
```

```
A = np.empty((nl, nc))
```

```
# le do teclado
```

```
for i in range(0, nl): # linhas vao de 0 a nl-1
```

```
    for j in range(0, nc): # colunas vao de 0 a nc-1
```

```
        s = 'Informe o elemento A[%d][%d]: ' % (i, j)
```

```
        A[i][j] = float(input(s))
```

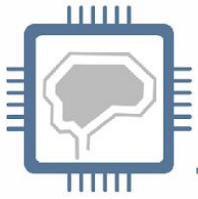
```
print("Imprime matriz...")
```

```
for i in range(0, nl):
```

```
    for j in range(0, nc):
```

```
        print('%5.1f' % (A[i][j]), end='')
```

```
    print() # imprime quebra de linha
```



Matrizes

Para cada linha da matriz, indicada pelo índice (i), percorre todas as suas colunas, indicadas pelo índice (j).

Assim, todos os elementos da matriz serão percorridos 1 a 1, linha por linha.

```
import numpy as np
```

```
# le dimensões da matriz
```

```
nl = int(input('Informe o número de linhas : '))
```

```
nc = int(input('Informe o número de colunas: '))
```

```
# cria matriz
```

```
A = np.empty((nl, nc))
```

```
# le do teclado
```

```
{ for i in range(0, nl): # linhas vao de 0 a nl-1
    for j in range(0, nc): # colunas vao de 0 a nc-1
        s = 'Informe o elemento A[%d][%d]: ' % (i, j)
        A[i][j] = float(input(s))
```

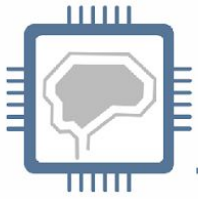
```
print("Imprime matriz...")
```

```
for i in range(0, nl):
```

```
    for j in range(0, nc):
```

```
        print('%5.1f' % (A[i][j]), end='')
```

```
    print() # imprime quebra de linha
```



Matrizes

Para cada elemento da matriz, o programa pede que o usuário informe um valor.

```
import numpy as np
```

```
# le dimensões da matriz
```

```
nl = int(input('Informe o número de linhas : '))
```

```
nc = int(input('Informe o número de colunas: '))
```

```
# cria matriz
```

```
A = np.empty((nl, nc))
```

```
# le do teclado
```

```
{ for i in range(0, nl): # linhas vao de 0 a nl-1
    for j in range(0, nc): # colunas vao de 0 a nc-1
        s = 'Informe o elemento A[%d][%d]: ' % (i, j)
        A[i][j] = float(input(s))
```

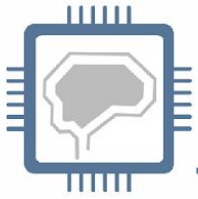
```
print("Imprime matriz...")
```

```
for i in range(0, nl):
```

```
    for j in range(0, nc):
```

```
        print('%5.1f' % (A[i][j]), end='')
```

```
    print() # imprime quebra de linha
```



Matrizes

Novamente, para cada linha da matriz, indicada pelo índice (i), percorre todas as suas colunas, indicadas pelo índice (j).

Imprime cada posição da matriz com a formatação de largura 5 e 1 casa decimal.

```
import numpy as np
```

```
# le dimensões da matriz
```

```
nl = int(input('Informe o número de linhas : '))
```

```
nc = int(input('Informe o número de colunas: '))
```

```
# cria matriz
```

```
A = np.empty((nl, nc))
```

```
# le do teclado
```

```
for i in range(0, nl): # linhas vao de 0 a nl-1
```

```
    for j in range(0, nc): # colunas vao de 0 a nc-1
```

```
        s = 'Informe o elemento A[%d][%d]: ' % (i, j)
```

```
        A[i][j] = float(input(s))
```

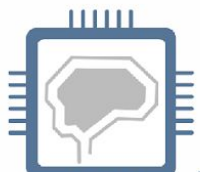
```
print("Imprime matriz...")
```

```
for i in range(0, nl):
```

```
    for j in range(0, nc):
```

```
        print('%5.1f' % (A[i][j]), end='')
```

```
    print() # imprime quebra de linha
```



Matrizes

```
import numpy as np
```

```
# le dimensões da matriz
```

```
nl = int(input('Informe o número de linhas : '))
```

```
nc = int(input('Informe o número de colunas: '))
```

```
# cria matriz
```

```
A = np.empty((nl, nc))
```

```
# le do teclado
```

```
for i in range(0, nl): # linhas vao de 0 a nl-1
```

```
    for j in range(0, nc): # colunas vao de 0 a nc-1
```

```
        s = 'Informe o elemento A[%d][%d]: ' % (i, j)
```

```
        A[i][j] = float(input(s))
```

```
print("Imprime matriz...")
```

```
for i in range(0, nl):
```

```
    for j in range(0, nc):
```

```
        print('%5.1f' % (A[i][j]), end='')
```

```
    print() # imprime quebra de linha
```

Saída do programa

```
Informe o número de linhas : 3
```

```
Informe o número de colunas: 4
```

```
Informe o elemento A[0][0]: 4
```

```
Informe o elemento A[0][1]: 1
```

```
Informe o elemento A[0][2]: 2.5
```

```
Informe o elemento A[0][3]: 2
```

```
Informe o elemento A[1][0]: -1
```

```
Informe o elemento A[1][1]: 0
```

```
Informe o elemento A[1][2]: 3
```

```
Informe o elemento A[1][3]: -2
```

```
Informe o elemento A[2][0]: 0.2
```

```
Informe o elemento A[2][1]: 6
```

```
Informe o elemento A[2][2]: 5
```

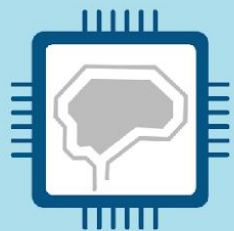
```
Informe o elemento A[2][3]: -0.1
```

```
Imprime matriz...
```

```
4.0  1.0  2.5  2.0
```

```
-1.0  0.0  3.0 -2.0
```

```
0.2  6.0  5.0 -0.1
```



Inteligência Artificial e Computacional

Profa.: Sabrina de A. Silveira